

«Talento Tech»

React JS

Clase 01



Clase N° 1 | Primeros Pasos con ReactJS

Índice

Primeros Pasos con ReactJS

Bienvenida y presentación del curso

Repaso de HTML, CSS y JavaScript: ¿Qué son y cómo funcionan?

¿Qué es ReactJS? Introducción y beneficios

La magia de React: virtual DOM, diffing y reconciliación

La Transpilación

Nociones Básicas de Nuestra Terminal

Windows

macOS

Linux

Configuración del entorno con Vite y Node.js

Un Vistazo a la Estructura del Proyecto

Creando y usando nuestro primer componente

Reflexión Final

Ejercicios prácticos:

Materiales y Recursos Adicionales:

Preguntas para Reflexionar:

Objetivos de la Clase:

- Revisar los fundamentos de HTML, CSS y JavaScript.
- Comprender el propósito de ReactJS y su lugar en el desarrollo web moderno.
- Configurar un proyecto inicial de React con Vite.
- Crear los primeros componentes funcionales usando JSX.

Bienvenida y presentación del curso

¡Bienvenidos al curso de **React**!



React

En este curso, exploraremos **React**, una de las bibliotecas más populares para construir interfaces de usuario en aplicaciones web. Aprenderemos desde los conceptos básicos, como componentes y estados, hasta temas más avanzados

como el manejo de ciclos de vida, hooks, y gestión del estado global.

Nuestro objetivo es ayudarlos a dominar React para que puedan desarrollar aplicaciones interactivas, modernas y escalables. Este curso está diseñado para que avancen paso a paso, con ejemplos prácticos y ejercicios dinámicos.

Repaso de HTML, CSS y JavaScript: ¿Qué son y cómo funcionan?

HTML, CSS y JavaScript son la base del desarrollo web y trabajan juntos para dar vida a las páginas que navegamos todos los días.



HTML (HyperText Markup Language) es el lenguaje que utilizamos para estructurar el contenido de una página web. Piensa en HTML como el esqueleto de la página: define qué elementos aparecen, como títulos, párrafos, imágenes o botones. Por ejemplo, si queremos mostrar un título principal y un botón, escribiríamos:

Ejemplo práctico:

1. HTML Básico

```
<h1>Hola Mundo</h1>
<button>Haz clic aquí</button>
```

Recordá que todo el contenido que queremos mostrar tiene que estar “envuelto” en etiquetas. Esto es importante porque le da estructura y significado a tu contenido. Además de que nos permite identificar cada parte para después poder darle estilo con CSS (colores, tamaños) o funcionalidad con JavaScript (que un botón haga algo al hacerle clic).



CSS (Cascading Style Sheets) es lo que usamos para estilizar y dar formato visual al contenido definido con HTML. Con CSS podemos cambiar colores, tamaños, tipografías, márgenes y mucho más, logrando que nuestras páginas sean atractivas y fáciles de usar. **Por ejemplo, para estilizar un botón, podríamos usar:**

2. CSS para diseño:

```
button {
  background-color: blue;
  color: white;
}
```

Recordá que el código en CSS se basa en dos conceptos fundamentales: la especificidad (qué regla es más importante y se aplica sobre otra) y la herencia (propiedades que los elementos "hijos" heredan de sus "padres").

Además, su sintaxis siempre va a ser la misma: Un **selector** (a qué elemento le damos estilo) y un **bloque de declaraciones** entre llaves (`{ }`). Dentro, vas a tener pares de propiedad: valor (qué característica cambiamos y qué valor le ponemos)



JavaScript es el lenguaje que añade interactividad a nuestras páginas. Es lo que hace que los botones respondan a los clics, que aparezcan mensajes o que cambien dinámicamente los datos en pantalla. JavaScript permite que las páginas sean dinámicas y funcionales, como en **este ejemplo donde mostramos una alerta al hacer clic en un botón:**

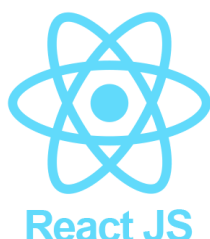
3. JavaScript interactivo:

```
document.querySelector('button').addEventListener('click', () => {
  alert('¡Botón clickeado!');
});
```

Además de estas funciones básicas, JavaScript también nos permite trabajar con estructuras más avanzadas como **arrays** y **objetos**, que son fundamentales para almacenar y manipular datos. **Por ejemplo:**

```
const numeros = [1, 2, 3, 4];
const dobles = numeros.map(num => num * 2); // [2, 4, 6, 8]
const persona = { nombre: "Ana", edad: 25 };
console.log(persona.nombre); // Ana
```

¿Qué es ReactJS? Introducción y beneficios



ReactJS es una biblioteca de JavaScript desarrollada por Meta (antes Facebook) que nos permite construir interfaces de usuario dinámicas de una forma sencilla y estructurada. Su principal característica es la capacidad de dividir la interfaz en pequeños bloques reutilizables llamados **componentes**, lo que mejora la organización y mantenimiento del código

Entre sus ventajas destacan:

- **Componentización:** Permite dividir interfaces complejas en piezas más pequeñas y reutilizables
- **Desempeño optimizado:** Gracias al **Virtual DOM**, una representación virtual que optimiza las actualizaciones, nuestras aplicaciones son más rápidas.
- **Gran ecosistema:** Documentación, herramientas, y una comunidad amplia que facilita el aprendizaje.

El Corazón de React: Componentes y JSX



En React, no escribimos HTML directamente. En su lugar, construimos nuestra interfaz usando **componentes**. Pensá en ellos como piezas de un LEGO: son bloques de código independientes y reutilizables que se encargan de una parte

específica de la UI (un botón, un formulario, un menú, etc.).

Para escribir lo que estos componentes deben mostrar en pantalla, usamos **JSX** (**JavaScript XML**). JSX es una extensión de JavaScript que nos permite escribir una sintaxis muy parecida a HTML dentro de nuestros archivos de JavaScript.

Por ejemplo, en lugar de escribir código JavaScript puro y más verboso como esto:

```
React.createElement('h1', {}, 'Hola Mundo');
```

Con JSX, podemos escribir de forma mucho más intuitiva:

```
<h1>Hola Mundo</h1>;
```

Ahora, si cada vez que un dato cambia (por ejemplo, el nombre de un usuario en pantalla) tuviésemos que redibujar toda la página, sería lentísimo. Aquí es donde React saca su as bajo la manga y demuestra por qué es tan eficiente.

La magia de React: virtual DOM, diffing y reconciliación

Para lograr un rendimiento óptimo, React no trabaja directamente sobre el DOM del navegador (la estructura real de la página). En su lugar, utiliza un concepto clave: el **Virtual DOM**.

Pensalo de esta manera:

1. **Virtual DOM (El borrador):** React crea una copia ligera de la estructura del DOM en la memoria. Es como un plano o un borrador de tu interfaz. Trabajar sobre esta copia es increíblemente rápido.
2. **Ocurre un cambio:** Cuando algo en mi sitio se actualiza, React no toca la página. En su lugar, crea un **segundo** Virtual DOM con los cambios ya aplicados.
3. **Diffing (Buscando las diferencias):** Ahora viene lo interesante. React compara el Virtual DOM nuevo con el anterior. Este proceso, llamado **diffing**, le permite encontrar de forma super rápida la lista exacta y mínima de cambios: "solo cambió el texto de este párrafo" o "se agregó un nuevo elemento a esta lista".
4. **Reconciliación (La actualización quirúrgica):** Una vez que React sabe exactamente qué cambió, inicia el proceso de **reconciliación**. Va al DOM real y aplica **únicamente** esas diferencias que encontró. No redibuja todo, solo actualiza las partes que de verdad lo necesitan.



Este mecanismo es el secreto detrás de la velocidad de React. Nos permite a los desarrolladores, escribir código como si la interfaz se creara de nuevo en cada cambio, mientras React se encarga por detrás de optimizar todo para que solo se realicen las operaciones estrictamente necesarias.

Esto nos lleva a la siguiente pregunta... si escribimos en JSX y React hace toda esta magia por detrás, ¿cómo es que el navegador entiende nuestro código JSX como si fuera JS?

Transpilación

¿Cómo entiende el navegador nuestro código?

La transpilación es el proceso de convertir el código que escribís con sintaxis moderna (como JSX) a una versión de JavaScript estándar que todos los navegadores puedan entender. Los navegadores **no entienden JSX directamente**.

Para esto, herramientas como **Babel** actúan como un traductor. Toman tu código de React y lo transforman en JavaScript compatible antes de que llegue al navegador. Babel ya viene incorporado en los generadores de proyecto como **Create React App** o los frameworks mas usados como **Vite** y [Next.js](#) (si arrancas un proyecto desde cero será necesario que los instales como dependencia)

Mira este ejemplo:

Código que vos escribís en React (JSX):

```
const App = () => <h1>Hola Mundo</h1>;
```

Código que el transpilador (Babel) genera:

```
const App = () => React.createElement("h1", null, "Hola Mundo");
```

Ese segundo código sí lo puede ejecutar cualquier navegador. Por eso, cuando trabajamos con React, siempre hay un proceso de "traducción" ocurriendo por detrás.

Nociones básicas de nuestra terminal

La terminal o consola es una herramienta de texto que nos permite darle órdenes directas a la computadora. Al principio puede parecer intimidante, pero con unos pocos comandos será necesario para ver que es súper rápida y fácil. Abrir la terminal es sencillo en cualquier sistema, acá te muestro las formas más directas.

Windows

En Windows tenés principalmente dos consolas: la tradicional **CMD** y la más moderna y potente, **PowerShell**. Ambas te sirven.

- **Método Rápido (Recomendado):**
 1. Hacé clic en el menú Inicio (o apretá la tecla de Windows).
 2. Empezá a escribir PowerShell o CMD.
 3. Apretá Enter cuando aparezca en los resultados.
- **Atajo de Teclado:**
 1. Apretá las teclas Windows + R para abrir la ventana "Ejecutar".
 2. Escribí cmd o powershell y dale a Enter.

macOS

En Mac, la aplicación se llama **Terminal**.

- **Método Rápido con Spotlight (Recomendado):**
 1. Apretá las teclas Command (⌘) + Barra espaciadora.
 2. Escribí `Terminal` en el buscador que aparece.
 3. Apretá Enter.
- **Desde el Finder:**
 1. Abrió el Finder.
 2. Andá a la carpeta Aplicaciones y luego a la subcarpeta Utilidades.
 3. Ahí adentro vas a encontrar `Terminal.app`.

Linux

La mayoría de las distribuciones de Linux (como Ubuntu, Mint, etc.) tienen un atajo de teclado universal.

- **Atajo de Teclado (El más común):**
 1. Simplemente apretá `Ctrl + Alt + T`.
- **Desde el Menú de Aplicaciones:**
 1. Abrió tu lanzador de aplicaciones.
 2. Buscá la palabra "Terminal" o "Consola". El ícono suele ser una pantalla negra.

Aunque los comandos pueden variar un poco entre sistemas operativos, la lógica es la misma: te movés entre carpetas (directorios) para ejecutar comandos en el lugar correcto.

Tarea	Comando en Linux / Mac	Comando en Windows (CMD)
Listar archivos y carpetas	<code>ls</code>	<code>dir</code>
Cambiar de directorio	<code>cd nombre_carpeta</code>	<code>cd nombre_carpeta</code>
Subir un nivel (a la carpeta padre)	<code>cd ..</code>	<code>cd ..</code>
Saber dónde estás parado	<code>pwd</code>	<code>cd</code> (sin argumentos)
Limpiar la pantalla	<code>clear</code>	<code>cls</code>

Tip clave: ¡Podés usar la tecla `Tab` para autocompletar nombres de archivos y carpetas! Es un salvavidas.

Configuración del entorno con Vite y Node.js



Antes de empezar, necesitamos configurar nuestro entorno. Usaremos **Node.js** que incluye: **Vite**, una herramienta moderna que crea y sirve nuestros proyectos de React de forma increíblemente rápida y npm (Node Package Manager) el gestor de paquetes.

1. **Instalar Node.js:** Descargalo desde su sitio oficial ingresando [ACA](#) o yendo a <https://nodejs.org/> e instálalo. Podés instalarlo entre otras formas, por el instalador de Windows (.msi). [Podes seguir este tutorial](#).
2. **Crear el proyecto con Vite:** Una vez instalado, creá una carpeta en el lugar que quieras y abrila con Visual Studio Code. En su terminal (ctrl + shift + ñ) ejecutá:

```
npm create vite@latest
```

Vite te hará algunas preguntas: dale un nombre a tu proyecto (ej: "app-TalentoTech"), seleccioná **React** como framework y **JavaScript** como variante.

3. **¡Ojo acá! Navegar a la carpeta del proyecto:** Una vez creado, la terminal no entra automáticamente a la carpeta. Tenés que hacerlo vos. Este paso es clave.

```
cd app-TalentoTech
```

Una vez dentro, por la terminal tenés que ver algo similar a esto:

```
PS C:\Users\USUARIO\Desktop\clasesReact\app-TalentoTech> 
```

4. **Instalar las dependencias:** Ya dentro de la carpeta, instalá todo lo que el proyecto necesita:

```
\app-TalentoTech> npm install
```

5. **Iniciar el servidor:** Para ver tu aplicación en el navegador, ejecutá:

```
t\app-TalentoTech> npm run dev
```

Si querés cortar la aplicación, en la terminal usa el atajo de teclado CTRL + C

Un Vistazo a la estructura del proyecto

Vite nos crea una estructura de archivos muy limpia. Antes de escribir nuestro propio código, entendamos las partes más importantes:

node_modules/: Es la carpeta donde se descargan todas las dependencias. Esta carpeta **nunca se sube a GitHub** porque pesa muchísimo. En su lugar, se guarda **el package.json** para que otro pueda instalar lo mismo con **npm install**.

public/ : Contiene archivos estáticos que no pasan por React ni por la transpilación de Vite. Ejemplo: imágenes, íconos, o el favicon.ico. **Todo lo que pongas acá estará disponible tal cual en la app** (ruta directa en la URL).

index.html: Es el único archivo HTML de toda tu aplicación. Si lo abris, vas a ver que tiene un `<div id="root"></div>`.

¡Acá es donde React va a "inyectar" toda nuestra aplicación!

package.json: Es el corazón del proyecto. Define el nombre, las versiones y, lo más importante, **las dependencias y los scripts** (como `npm run dev`).

/src/main.jsx: Acá es donde le decimos a React: *"buscá el div con id 'root' en el 'index.html' y renderiza nuestro componente principal adentro"*. Tenemos que estar seguros al momento de modificar este archivo. Tiene su propio archivo CSS

/src/App.jsx ¡Este es nuestro primer componente! Es el componente principal que actúa como contenedor para todos los demás componentes que vayamos creando. Tiene su propio archivo CSS

.gitignore: Lista de archivos/carpetas que **no queremos subir al repositorio GitHub**. Ejemplo: node_modules/, archivos de caché, etc.

eslint.config.js: Configuración de **ESLint**, una herramienta que revisa tu código para que mantengas buenas prácticas y un estilo consistente.

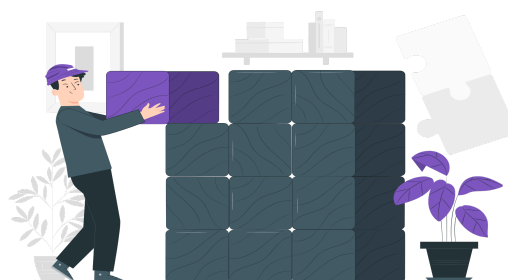
package-lock.json: Complemento de package.json que guarda las versiones exactas de todas las dependencias instaladas para que el proyecto funcione igual en cualquier computadora.

vite.config.js: Archivo de configuración de **Vite**, la herramienta que levanta el servidor de desarrollo y empaqueta tu app.



Creando y usando nuestro primer componente

Ahora sí, ¡a programar! Cuando creamos el proyecto, Vite nos regaló un componente `App.jsx` con un ejemplo de un contador. Es un buen punto de partida, pero ahora vamos a limpiarlo para empezar a construir nuestra propia aplicación desde cero. Nuestro objetivo es crear un componente simple de bienvenida y mostrarlo en la pantalla principal.



Paso 1: Crear nuestro componente "hijo" (`Bienvenida.jsx`)

Primero, vamos a crear el componente que queremos mostrar. Por convención, los nombres de los archivos de componentes empiezan con mayúscula.

1. **Creá un nuevo archivo:** Dentro de la carpeta `/src`, creá un archivo llamado `Bienvenida.jsx`. Cuando retornamos un solo elemento JSX, como por ejemplo un `<h1>` podemos hacerlo en una sola línea como se muestra a continuación.

Escribí el componente: Un componente es simplemente una función de JavaScript que devuelve JSX. Copiá el siguiente código en tu nuevo archivo y guardalo.

 (Recordá que podés tener activado el guardado automático en VS Code, yendo a "Archivo" en la barra de navegación y asegurándote de tener marcada la opción "Autoguardado")

```
import React from "react";

function Bienvenida() {
  return <h1>¡Bienvenidos a mi curso!</h1>;
}

export default Bienvenida; // Clave para poder usarlo en App.jsx u otros archivos
```

Pero si queremos retornar varios elementos, necesitamos envolverlos dentro de un contenedor como un `<div>` o un `<>` (fragment).

```
function Bienvenida(){
  return (
    <div>
      <h1>¡Bienvenidos a mi curso!</h1>
      <p>Vamos por mas 🥰</p>
    </div>
  );// Continua
```

```

}
export default Bienvenida;

```

Un solo elemento: se puede devolver directamente.

Varios elementos: React exige que estén dentro de un único contenedor.

- Ese contenedor puede ser un `<div>` o un Fragmento (`<> ... </>`).
- Los fragmentos son útiles cuando no querés agregar un div extra al HTML.

Paso 2: Limpiar y preparar nuestro componente principal (App.jsx)

Ahora, vamos al archivo `/src/App.jsx`. Este es nuestro componente “padre” o principal. Actualmente, contiene la lógica del contador que Vite creó. y veíamos en pantalla al correr nuestro código. Vamos a limpiarlo para mostrar únicamente nuestro componente Bienvenida.

- Abrí `/src/App.jsx`. Vas a ver un código similar a este:

```

// Código original de Vite que lo vamos a borrar!
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

function App() {
  const [count, setCount] = useState(0)
  // ... código JSX con logos, botones y textos
}
export default App

```

- **Reemplazá todo el contenido** del archivo `App.jsx` con este código base. Estamos quitando toda la lógica del contador (`useState`), las importaciones de los logos y el JSX de ejemplo para tener un lienzo limpio.

```

// En /src/App.jsx
import './App.css';
function App() {
  return (
    <div>
      {/* Acá vamos a poner nuestro componente */}
    </div>
  );
}
export default App;

```

Ahora tenemos un componente `App` vacío y listo para recibir a otros componentes adentro.

Paso 3: Integrar Bienvenida dentro de App

Con nuestro componente principal ya limpio, vamos a "invocar" o "renderizar" nuestro componente Bienvenida adentro.

1. Importá el componente Bienvenida al principio de App.jsx. Así, App sabe que existe y puede usarlo.
2. Usá el componente Bienvenida dentro del div del return, como si fuera una etiqueta HTML.

El código final de tu /src/App.jsx debería verse así:

```
// En /src/App.jsx
import Bienvenida from './Bienvenida'; // 1. Importamos nuestro componente
import './App.css';

function App() {
  return (
    <div>
      {/* 2. Lo usamos como si fuera una etiqueta HTML */}
      <Bienvenida />
      <p>Este es mi primer componente montado en App.jsx</p>
    </div>
  );
}

export default App;
```

¡Listo! Si volvéis a tu navegador, deberías ver el saludo de tu nuevo componente. Acabás de crear tu primer componente y lo integramos en el flujo principal de la aplicación, ¡felicitaciones!

Paso 4: Montar App dentro de main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```


Reflexión Final

¡Muy bien! Ya calentamos motores con un repaso técnico y conocimos un poco de qué trata React. Ahora contás con la experiencia para crear proyectos React y poder trabajar sobre un proyecto creado con Vite. A partir de acá será nuestra tarea ir construyendo en necesidad un producto digital que luzca moderno y sencillo para el uso en el mercado y fácil para los usuarios.

Ejercicios prácticos:

Vamos a practicar la división de una interfaz en pequeños componentes, que es el corazón de React. Vamos a armar la estructura de un posteo simple. Puede ser nuestra presentación.

Objetivo: Crear tres componentes separados y usarlos dentro de App.jsx para formar una única pieza de interfaz.

Paso a paso:

1. **Preparar el terreno:**
 - Creá un proyecto nuevo con Vite (npm create vite@latest).
 - Navegá a la carpeta, instalá las dependencias (npm install) y levantá el servidor (npm run dev).
 - Limpiá el archivo /src/App.jsx como hicimos en la clase, dejándolo listo para recibir nuestros nuevos componentes.
2. **Crear los componentes "hijos":** Dentro de la carpeta /src creá:
 - Un archivo llamado **Encabezado.jsx** que represente a un título h1
 - Otro archivo llamado **CuerpoPosteo.jsx** que contenga un párrafo o alguna descripción que te interese
 - Otro llamado **PieDePosteo.jsx** para generar el footer de la página
3. **Ensamblar todo en App.jsx (el componente "padre"):** Andá a tu archivo /src/App.jsx e importá los tres componentes que acabás de crear al principio del archivo.
4. Usalos dentro del return de la función App, en orden.
5. Si mirás el navegador, deberías ver tu posteo completo. Con esto, el concepto de dividir para conquistar queda mucho más claro. ¡A meterle!

Materiales y Recursos Adicionales:

- [Documentación oficial de React](#)
 - [Guía de instalación de Node.js](#)
 - [Guía oficial de Vite](#)
-

Preguntas para Reflexionar:

- ¿Qué relación tienen HTML, CSS y JavaScript en un proyecto?
 - ¿Qué ventajas crees que aporta React frente a JavaScript puro?
 - ¿Cómo podrías mejorar la reutilización de código en un proyecto?
-

Próximos Pasos:

- Reforzar el uso y las ventajas de **JSX**.
- Repasar los **conceptos de JavaScript importantes** para el curso
- Entender las **props** y sus diferentes técnicas.
- **Organizar nuestros componentes** de forma prolija y escalable.
- Diferenciar entre componentes de **presentación** y componentes **contenedores**.



Buenos Aires
aprende
Agencia de Actividades para el Futuro

BA Buenos
Aires
Ciudad